

GitHub Pages 粒子演算法專案開發計畫書

適用於 AI Agent 自動化開發與落地之高規格工程規格書

🤖 Instructions for AI Agent:

- Execute this project sequentially based on the steps outlined in this document.
- Complete each sub-task step-by-step. Maintain clean code structure and type safety.
- Ensure every file is strictly formatted according to the specifications.
- Verify that the build command succeeds before moving to the next task.

一、專案檔案結構 (FILE STRUCTURE TREE)

```
particle-lab/ ├── .github/ | ├── workflows/ | ├── deploy.yml | ├── public/ | ├── favicon.ic
```

二、技術選型規格 (TECHNICAL ARCHITECTURE)

模組分類	技術選型	執行與建置說明
渲染引擎	Three.js (WebGL 2.0)	使用 InstancedMesh 或 Points 實現 100,000 粒子高效能渲染。
開發建置環境	Vite + TypeScript	設定 base: './' 確保 GitHub Pages 靜態路徑解析正確。
UI 控制面板	Tweakpane + Stats.js	提供即時參數調整介面與即時 FPS/記憶體監控面板。
自動化部署	GitHub Actions	配置 actions/deploy-pages@v4 觸發自動化 CI/CD 流程。

三、階段式執行管道 (EXECUTION PIPELINE)

Phase 1: Project Initialization & Baseline

任務點 1.1 - 1.4

- 1.1: 初始化 Vite + TypeScript 專案。
- 1.2: 安裝依賴：three, @types/three, tweakpane, stats.js。
- 1.3: 配置 vite.config.ts 設定相對路徑 base: './'。
- 1.4: 建立 .github/workflows/deploy.yml 配置 GitHub Pages 自動部署。

✓ 驗證標準：執行 `npm run build` 成功產生無錯 dist/ 產物。

Phase 2: Core Interfaces & Rendering System

任務點 2.1 - 2.3

- 2.1: 於 BaseAlgorithm.ts 定義粒子數據結構 (Position, Velocity, Color)。
- 2.2: 實作 ParticleRenderer.ts 高效能粒子渲染器。
- 2.3: 設定 main.ts 包含 Canvas 自動 Resize 與 requestAnimationFrame 渲染迴圈。

✓ 驗證標準：畫面上成功渲染出 10,000 顆靜態/隨機散佈之粒子。

Phase 3: Algorithms Implementation

任務點 3.1 - 3.2

- 3.1: 實作 BoidsAlgorithm.ts (Separation、Alignment、Cohesion 三大群聚法則)。
- 3.2: 實作 ParticleSwarmOptimization.ts (PSO 粒子群優化尋優演算法)。

✓ 驗證標準：切換模式時粒子能根據演算法公式流暢更新運動軌跡。

Phase 4 & 5: UI Integration & Deployment

任務點 4.1 - 5.3

- 4.1: 整合 Tweakpane 綁定演算法權重參數與粒子數量。
- 5.1: 執行全專案 TypeScript 型別檢查 tsc --noEmit。
- 5.2: Commit 所有檔案並 Push 至 GitHub 倉庫 main 分支觸發部署。

✓ 驗證標準：GitHub Actions 部署成功且網頁功能運作正常。

四、核心演算法數學模型 (ALGORITHM SPECIFICATIONS)

1. Boids Flocking Formula (鳥群模擬)

$$F_{total} = w_s \times F_{separation} + w_a \times F_{alignment} + w_c \times F_{cohesion}$$

- Separation: $F_s = \sum (p_{self} - p_{other}) / ||p_{self} - p_{other}||^2$
- Alignment: $F_a = (1/N) \sum v_{other} - v_{self}$ | Cohesion: $F_c = ((1/N) \sum p_{other}) - p_{self}$

2. Particle Swarm Optimization (PSO 粒子群尋優)

$$v_i^d = w \times v_i^d + c_1 r_1 (pbest_i^d - x_i^d) + c_2 r_2 (gbest^d - x_i^d)$$

- Position Update: $x_i^d = x_i^d + v_i^d$ (動態尋找函數地形極值點)